MongoDB DATABASE DESIGN

Chat Application — Database Schema Reference

Version 1.0 | May 2026 | Database: chatapp_db

1. Database Overview

The Chat Application uses **MongoDB** as its primary database. MongoDB's flexible document model is well-suited for chat apps where message structures can vary (text, image, file) and participant arrays fit naturally as embedded documents.

Collection	Documents Stored	Approx. Size
`users`	One document per registered user	Small — grows with users
`chats`	One document per conversation	Medium — one per user pair
`messages`	One document per message sent	Large — grows with usage
`sessions`	One document per active login session	Small — auto-expires

2. Collections Schema

■ users

Stores all registered user accounts including credentials and profile information.

Field	Type	Description
`_id`	ObjectId	Auto-generated unique identifier (Primary Key)
`name`	String	Full name of the user (max 100 chars)
`email`	String	Unique registered email address (max 150 chars)
`password_hash`	String	Bcrypt hashed password — never store plain text
`profile_photo`	String	CDN URL of the user's profile photo
`is_online`	Boolean	Current online/offline status (default: false)
`last_seen`	Date	Timestamp of last activity
`created_at`	Date	Account creation timestamp

Field	Type	Description
updated_at	Date	Last profile update timestamp

Sample Document:

```
{
  "_id":          ObjectId("64abc1230000000000000001"),
  "name":         "John Doe",
  "email":        "john@example.com",
  "password_hash":"$2b$10$KIXv...",
  "profile_photo":"https://cdn.chatapp.com/photos/u123.jpg",
  "is_online":    false,
  "last_seen":    ISODate("2026-05-31T10:30:00Z"),
  "created_at":   ISODate("2026-01-01T00:00:00Z"),
  "updated_at":   ISODate("2026-05-31T10:30:00Z")
}
```

Indexes:

Index Field	Index Type	Purpose
email	Unique Index	Fast lookup during login; prevents duplicate accounts
name	Text Index	Enables full-text search for finding users

■ chats

Stores one document per conversation between two users. Also caches the last message and unread counts to avoid extra queries when loading the Chat List Screen.

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier (Primary Key)
participants	ObjectId[ ]	Array of exactly 2 user _id references
last_message	Object	Embedded: { text, sender_id, timestamp } — cached for chat list
unread_counts	Object	Map of userId → unread count e.g. { 'u123': 0, 'u456': 3 }
created_at	Date	When the conversation was first started
updated_at	Date	Updated on every new message — used to sort chat list

Sample Document:

```
{
  "_id":          ObjectId("64abc4560000000000000002"),
  "participants": [
    ObjectId("64abc1230000000000000001"),
    ObjectId("64abc7890000000000000003")
  ],
  "last_message": {
    "text":      "Hey, how are you?",
    "sender_id": ObjectId("64abc1230000000000000001"),
    "timestamp": ISODate("2026-05-31T10:30:00Z")
  },
  "unread_counts": { "64abc123": 0, "64abc789": 3 },
  "created_at":   ISODate("2026-01-15T00:00:00Z"),
  "updated_at":   ISODate("2026-05-31T10:30:00Z")
}
```

Indexes:

Index Field	Index Type	Purpose
`participants`	Index	Fetch all chats for a given user ID quickly
`updated_at (desc)`	Descending Index	Sort Chat List by most recent message
`participants (compound)`	Unique Compound	Prevent duplicate chat between same two users

■ *The last_message and unread_counts fields are denormalized (duplicated) intentionally to avoid expensive queries when rendering the Chat List Screen.*

✉■ messages

Stores every individual message sent in any conversation. This will be the largest collection in the database and benefits most from proper indexing.

Field	Type	Description
`_id`	ObjectId	Auto-generated unique identifier (Primary Key)
`chat_id`	ObjectId	Reference to parent chats._id
`sender_id`	ObjectId	Reference to the user who sent the message
`text`	String	Message text content (null for media messages)
`type`	String	Message type: 'text', 'image', or 'file'
`file_url`	String	CDN URL of the uploaded file/image (null for text)
`file_name`	String	Original filename of the attachment
`file_size_kb`	Number	File size in kilobytes (max 10240 = 10 MB)

Field	Type	Description
`is_read`	Boolean	Whether the recipient has read this message
`created_at`	Date	Timestamp when the message was sent

Sample Document (Text Message):

```
{
  "_id":         ObjectId("64abcdef0000000000000004"),
  "chat_id":     ObjectId("64abc4560000000000000002"),
  "sender_id":   ObjectId("64abc1230000000000000001"),
  "text":        "Hey, how are you?",
  "type":        "text",
  "file_url":    null,
  "file_name":   null,
  "file_size_kb":null,
  "is_read":     false,
  "created_at":  ISODate("2026-05-31T10:30:00Z")
}
```

Sample Document (Image Message):

```
{
  "_id":         ObjectId("64abcdf10000000000000005"),
  "chat_id":     ObjectId("64abc4560000000000000002"),
  "sender_id":   ObjectId("64abc1230000000000000001"),
  "text":        null,
  "type":        "image",
  "file_url":    "https://cdn.chatapp.com/uploads/img_abc.jpg",
  "file_name":   "photo.jpg",
  "file_size_kb":248,
  "is_read":     false,
  "created_at":  ISODate("2026-05-31T10:35:00Z")
}
```

Indexes:

Index Field	Index Type	Purpose
`chat_id`	Index	Fetch all messages of a conversation
`chat_id + created_at`	Compound Index	Paginated message loading (oldest/newest first)
`sender_id`	Index	Fetch all messages sent by a user
`chat_id + is_read`	Compound Index	Count unread messages per chat efficiently

■ sessions

Stores active login sessions. Uses a MongoDB TTL index to automatically delete expired sessions without any manual cleanup required.

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier (Primary Key)
user_id	ObjectId	Reference to the logged-in user's _id
token	String	JWT Bearer token string
expires_at	Date	Expiry timestamp — TTL index deletes document after this time
created_at	Date	Login timestamp

Sample Document:

```
{
  "_id":       ObjectId("64abcef00000000000000006"),
  "user_id":   ObjectId("64abc1230000000000000001"),
  "token":     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_at":ISODate("2026-06-01T10:30:00Z"),
  "created_at":ISODate("2026-05-31T10:30:00Z")
}
```

Indexes:

Index Field	Index Type	Purpose
token	Unique Index	Fast token validation on every authenticated request
expires_at	TTL Index	Auto-deletes expired session documents (expireAfterSeconds: 0)

3. Collection Relationships

MongoDB uses references (ObjectId) to link documents across collections, similar to foreign keys in SQL databases.

From Collection	Field	References	Relationship
chats	participants[ ]	users._id	Many-to-Many (2 users per chat)
messages	chat_id	chats._id	Many-to-One (many messages per chat)
messages	sender_id	users._id	Many-to-One (many messages per user)
sessions	user_id	users._id	Many-to-One (many sessions per user)

4. Redis — Real-Time Cache

Redis is used alongside MongoDB for features that require millisecond response times. These values are never persisted long-term — they live only in memory.

Redis Key Pattern	Value	TTL	Purpose
`online:<userId>`	true / false	5 min	Live online/offline status indicator
`typing:<chatId>:<userId>`	1	3 seconds	Typing indicator — auto-expires
`session:<token>`	userId	30 min	Fast token validation — avoids DB hit
`unread:<userId>:<chatId>`	count (int)	No expiry	Unread message badge count

5. Recommended Tech Stack

Layer	Technology	Notes
Database	**MongoDB Atlas**	Free M0 cluster for development; scales easily
Cache	**Redis (Upstash)**	Free tier available; serverless Redis
File Storage	**Cloudinary / AWS S3**	Store photos & attachments; return CDN URL
ODM Library	**Mongoose (Node.js)**	Schema validation + easy MongoDB integration
Backend	**Node.js + Express.js**	REST APIs + WebSocket (socket.io)
Real-time	**Socket.IO**	Built on WebSocket; handles reconnection
Auth	**JWT + bcrypt**	Stateless token auth; bcrypt for password hashing

6. Revision History

Version	Date	Author	Description
1.0	May 2026	Development Team	Initial MongoDB schema design for Chat Application